UNIVERSIDAD NACIONAL EVANGÉLICA · PROGRAMACIÓN II

# Publicación de proyectos JavaFX y pruebas de software

Análisis DevOps del proyecto SIGE · Elian Castro · 25 octubre 2026

Informe de investigación, estado de automatización y plan de evidencias. Se identifican expresamente las pruebas y publicaciones que siguen pendientes.

# DevOps, publicación y pruebas - SIGE

## 1. Alcance del análisis

SIGE es una aplicación Java 21 que conserva una interfaz de escritorio JavaFX y añade SIGE Web, un servidor Java HTTP con páginas adaptables que reutiliza MySQL. El repositorio remoto configurado es `https://github.com/Invicto1013/Sistema-de-Gestion-Escolar-con-Java.git`. El proyecto ya tiene `pom.xml`, `README_WEB.md`, `Dockerfile`, `GUIA_DESPLIEGUE_NUBE.md`, ejemplo de configuración y reglas Git para omitir `db.properties` y `target/`.

La web no es JavaFX ejecutándose en Chrome. El componente publicado como servicio es el backend web ligero. MySQL y sus credenciales deben vivir fuera del repositorio y del contenedor.

## 2. DevOps en palabras simples

DevOps combina prácticas de desarrollo (Dev) y operación (Ops) para entregar cambios pequeños, repetibles y verificables. CI (integración continua) compila y prueba cada cambio integrado. Entrega continua (continuous delivery) mantiene una versión validada lista para publicación manual. Despliegue continuo (continuous deployment) publica automáticamente cada cambio que supera las reglas. El workflow SIGE implementa CI y genera artefactos; no realiza despliegue automático a producción ni crea una Release.

Git guarda el historial distribuido. GitHub y GitLab alojan repositorios Git y añaden revisión y automatización. `commit` representa un cambio identificado; una rama permite trabajar aislado; `merge` integra cambios revisados. GitHub Actions y GitLab CI/CD son motores de CI integrados a su plataforma.

## 3. Cuadro comparativo de herramientas

| Herramienta | Función | Etapa DevOps | Ejemplo en SIGE |
|---|---|---|---|
| Git | Historial, ramas y commits locales distribuidos. | Planificar, desarrollar | Ramas para cambios de catálogo; commits identifican fases. |
| GitHub / GitLab | Alojamiento remoto, revisión de cambios, incidencias y Releases. | Colaborar, publicar | Remoto GitHub ya configurado; PR y Release todavía requieren evidencia/acción. GitLab es alternativa con `.gitlab-ci.yml`. |
| GitHub Actions | Workflows YAML disparados por push, PR o manualmente; runner ejecuta jobs y steps. | Integrar, probar, empaquetar | `.github/workflows/ci.yml`: verify y artefactos Windows/Linux/macOS. |
| GitLab CI/CD | Pipeline definido por jobs y stages en `.gitlab-ci.yml`, ejecutado por runners. | Integrar, probar, desplegar | Alternativa equivalente; no se configura además de Actions para evitar dos pipelines redundantes. |
| Jenkins | Servidor/automatizador extensible con pipelines `Jenkinsfile`, agentes, stages y steps. | Integrar, probar, entregar | Útil si una institución quiere administrar runners propios; no es necesario en la versión actual. |
| CircleCI | CI en nube con jobs y workflows; permite cache, workspaces y artifacts. | Integrar, acelerar, compartir | Alternativa para Maven; cachea dependencias y conserva artefactos por ejecución. |
| Docker | Dockerfile describe cómo construir imagen; contenedor ejecuta esa imagen aislada. | Empaquetar, desplegar | Dockerfile existente compila/ejecuta SIGE Web; no empaqueta la app gráfica JavaFX. |
| Kubernetes | Orquesta contenedores; un Pod ejecuta uno o más contenedores; clúster agrupa nodos. | Operar, escalar | Excesivo para demo pequeña; en producción podría correr varias réplicas web. Rolling update sustituye Pods gradualmente. |
| Terraform | Infraestructura como código declarativa; `plan` muestra cambios antes de `apply`. | Aprovisionar | Futuro: definir red, servicio y base administrada, sin guardar state/secretos en Git. |
| Ansible | Playbooks YAML automatizan configuración de servidores. | Configurar, desplegar | Futuro: instalar runtime, certificados y proxy en servidor institucional. |

| | | | |
|---|---|---|---|
| Nginx | Servidor HTTP y proxy inverso que reenvía tráfico al backend; puede terminar TLS. | Entrada, operación | Futuro: delante de SIGE Web para HTTPS si el proveedor no entrega TLS. |
| Prometheus / PromQL | Prometheus recoge métricas de series temporales; PromQL las consulta. | Monitorear | Futuro: tiempo de respuesta HTTP, errores y uso de JVM; SIGE hoy no expone endpoint de métricas. |
| Grafana | Paneles consultan y muestran datos de fuentes de métricas/logs. | Monitorear | Futuro: panel operacional con latencia, disponibilidad y fallos de conexión DB. |
| ELK (Elastic Stack) | Logstash ingiere/transforma, Elasticsearch indexa y Kibana busca/visualiza eventos. | Observar, diagnosticar | Futuro: centralizar logs de errores, sin incluir contraseñas, tokens ni datos académicos sensibles. |
| SonarQube | Analiza estáticamente código y evalúa reglas de calidad/seguridad. | Verificar, calidad | Siguiente mejora opcional después de que CI y pruebas base estén estables. |
| HashiCorp Vault | Centraliza secretos, credenciales, certificados y políticas de acceso. | Proteger, operar | Alternativa organizacional para DB/2FA/SMTP; por ahora variables seguras del proveedor y `db.properties` local ignorado. |

**GitHub Actions: elementos**

Un **workflow** es el proceso YAML guardado en `.github/workflows/`. Un **evento** (`push`, `pull_request`, manual) lo inicia. Un **job** corre en un runner; **steps** ejecutan comandos o acciones. El workflow de SIGE usa `setup-java` con Java 21, `mvn clean verify` y luego copia dependencias runtime y archivos de ejecución a un artefacto por sistema operativo. Si una prueba falla, `verify` devuelve error y ese job no llega a la etapa de empaquetado/subida.

Cache y artifact no significan lo mismo: la cache evita volver a descargar dependencias sin cambiar el resultado lógico del build; artifact conserva el resultado generado para descargarlo. En el workflow de SIGE se activa cache de Maven y se sube la carpeta de distribución como artifact temporal.

# 4. Diagrama de flujo

Código + JUnit -> GitHub -> Actions: compilar y probar -> paquete por SO -> Release manual -> Docker -> monitoreo

# 5. Pruebas de software y JUnit

**Verificación** pregunta si implementamos correctamente los requisitos; incluye revisión de código y ejecución de pruebas. **Validación** pregunta si el sistema realmente resuelve lo que necesita la escuela. Una **prueba unitaria** aísla una función pequeña; una **prueba de integración** comprueba el intercambio entre capas/servicios, como DAO con MySQL. **Caja negra** deriva casos desde entradas y resultados esperados sin depender del código interno. Las **particiones de equivalencia** agrupan entradas que deberían comportarse igual; se elige al menos una representante por grupo válido e inválido. JUnit ejecuta aserciones como parte del ciclo Maven.

En `EscalaCalificacionesTest` se prueba la regla real `EscalaCalificaciones`: la nota ponderada (30% + 30% + 40%), el redondeo, los límites de letra y el umbral 70 para aprobar. Incluye entradas válidas y valores límite. Son unit tests puros; no prueban MySQL ni la UI.

| ID | Tipo | Módulo | Entrada | Esperado | Estado local |
|---|---|---|---|---|---|
| UT-01 | Unitaria | Escala de calificación | 90, 80, 70 | Final 80.00 | Pendiente de `mvn verify` en GitHub/local con Maven resolviendo JUnit. |
| UT-02 | Unitaria | Escala de letras | 90 y 89.99 | A y B | Incluida en JUnit; aún sin ejecución CI registrada. |
| UT-03 | Límite | Aprobación | 70 y 69.99 | Aprobado / no aprobado | Incluida en JUnit; aún sin ejecución CI registrada. |

# 6. Distribución, deploy y evaluación del estado real

**Hecho en el repositorio:** el remote GitHub está configurado; SIGE incluye JavaFX, servidor web, Dockerfile y guías de despliegue; se agregó workflow CI multiplataforma; se agregaron tres métodos de prueba JUnit y launchers de distribución. Cada ejecución de Actions, después de subir estos cambios, debe producir artifacts descargables por sistema operativo.

**Aún no demostrado:** el workflow no ha corrido en GitHub porque estos cambios no se han enviado; no existe evidencia de un artifact descargable generado por Actions, Release pública 1.0, captura de pipeline, instalación en una computadora ajena o revisión por compañero. No se registra aquí un test exitoso que no se haya ejecutado. El intento local de Maven fue bloqueado al resolver Surefire/JUnit por error de certificado TLS de Maven Central; no se deshabilitó la validación TLS.

**Integración manual sugerida en clase (datos ficticios):**

| ID | Tipo | Módulo | Entrada | Resultado esperado | Resultado obtenido / estado |
|---|---|---|---|---|---|
| IT-01 | Integración | Registro -> DAO -> MySQL | Crear estudiante de prueba, no real | Se guarda cuenta y perfil, luego aparece al consultar | Completar en PC de prueba; evidencia pendiente |
| IT-02 | Integración | Calificación -> DAO -> boleta | Parciales 90/80 y examen 70 | Final 80.00 aparece al consultar boleta | Completar en PC de prueba; evidencia pendiente |
| BB-01 | Caja negra / equivalencia | Registro | Correo válido Gmail / formato incorrecto / correo duplicado | Acepta válido; rechaza formato y duplicado | Completar y adjuntar captura |

Intercambiar con un compañero el ZIP/Release y el repositorio. No usar información real ni borrar datos; crear y limpiar solo registros de prueba con autorización. Anotar nombre del evaluador, commit/versión, pasos, resultado, error y recomendación. El compañero debe agregar dos casos unitarios (incluido uno de caja negra) y ejecutar dos integraciones.

# 7. Tres herramientas recomendadas para SIGE

1. **Git + GitHub**: conservar historial, revisar cambios y distribuir Releases.
2. **GitHub Actions**: compilar, ejecutar JUnit y generar paquetes cada push/PR sin configurar un servidor Jenkins.
3. **JUnit 5**: verificar rápidamente reglas de negocio como notas y validaciones sin conectarse a MySQL.

Docker ya se aprovecha para la parte web; JDK `jpackage` conviene evaluar después para un instalador JavaFX verdaderamente autocontenido. `jpackage` genera paquetes específicos de cada sistema operativo, por lo que debe construirse en runner Windows/macOS/Linux correspondiente. El artifact de Maven no se debe llamar instalador certificado: debe probarse en otra computadora con JDK 21 y MySQL configurado.

# 8. Pasos para cerrar la entrega

1. Revisar cambios y ejecutar `mvn clean verify` en el equipo con Maven Central funcional.
2. Guardar `.github/workflows/ci.yml`, `pom.xml`, pruebas y scripts en GitHub; comprobar el estado de cada matriz OS en Actions y capturar pantalla.
3. Descargar el artifact Windows y probarlo desde otra computadora sin rutas de la PC original; crear `db.properties` local con base de pruebas.
4. Crear tag `v1.0.0` y GitHub Release cuando la versión sea estable; adjuntar los paquetes y conectar el enlace en README/PDF.
5. Hacer práctica con compañero y completar el acta con resultados reales.
6. Grabar video de 3-5 minutos mostrando arquitectura, ejecución, CI, pruebas y Release; añadir URL al informe final.

# Referencias oficiales


• GitHub Actions Java/Maven: https://docs.github.com/en/actions/tutorials/build-and-test-code/java-with-maven
• Conceptos y sintaxis GitHub Actions: https://docs.github.com/en/actions/concepts/workflows-and-actions/workflows
• GitLab pipelines: https://docs.gitlab.com/ci/pipelines/
• Jenkins Pipeline: https://www.jenkins.io/doc/book/pipeline/
• CircleCI jobs/artifacts: https://circleci.com/docs/guides/orchestrate/jobs-steps/
• CircleCI caches: https://circleci.com/docs/guides/optimize/caching/
• Dockerfile: https://docs.docker.com/get-started/docker-concepts/building-images/writing-a-dockerfile

- Kubernetes rolling update: https://kubernetes.io/docs/tutorials/kubernetes-basics/update/update-intro/
- Terraform workflow: https://developer.hashicorp.com/terraform/intro/core-workflow
- Ansible playbooks: https://docs.ansible.com/projects/ansible/latest/playbook_guide/playbooks_intro.html
- Nginx proxy: https://nginx.org/en/docs/beginners_guide.html
- Prometheus y PromQL: https://prometheus.io/docs/introduction/overview/ y https://prometheus.io/docs/prometheus/latest/querying/basics/
- Grafana dashboards: https://grafana.com/docs/grafana/latest/visualizations/
- Elastic Stack: https://www.elastic.co/docs/get-started/the-stack
- SonarQube analysis: https://docs.sonarsource.com/sonarqube-community-build/analyzing-source-code/analysis-overview
- Vault: https://developer.hashicorp.com/vault/docs/about-vault/what-is-vault
- JUnit: https://docs.junit.org/5.4.2/user-guide/index.html
- JavaFX: https://openjfx.io/openjfx-docs/
- Java 21 jpackage: https://docs.oracle.com/en/java/javase/21/docs/specs/man/jpackage.html